The University of Jordan

King Abdullah II College for Information Technology

Department of Computer Information Systems

SECURITY OF WEB APPLICATION

T-CUP

Supervised by: Dr. Mohammed Almaiah

Name	University id	Section
Ahmad Farhan Taha	0227187	1
Anas Ziad Shelbaya	0225971	1
Saleh Nawahda	0211013	1
Hamza Naser	0211049	1

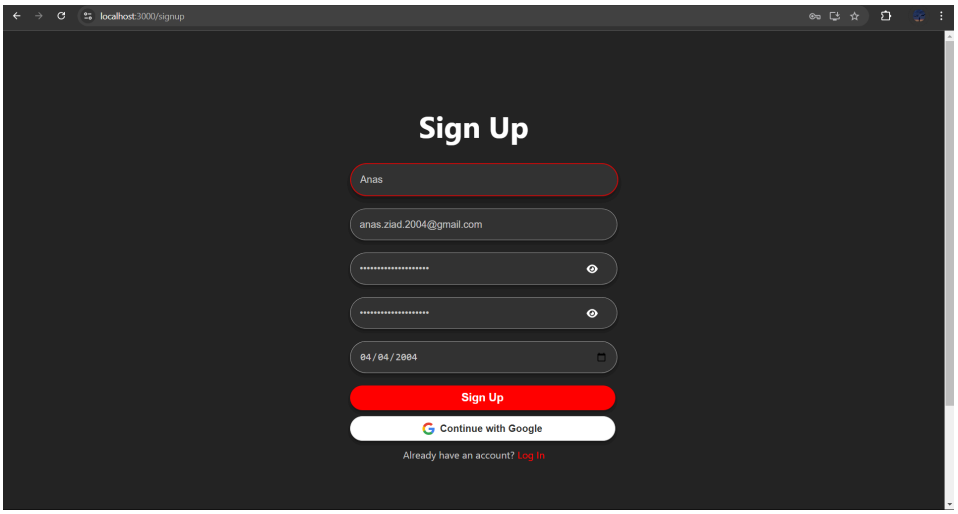
Layout Design

1- Registration Page

The registration page has Input fields for :

- **username**
- **Email**
- **Password**
- **Password confirmation** with validation
- **Date of Birth**

This page also has the option of signing up using the google account.

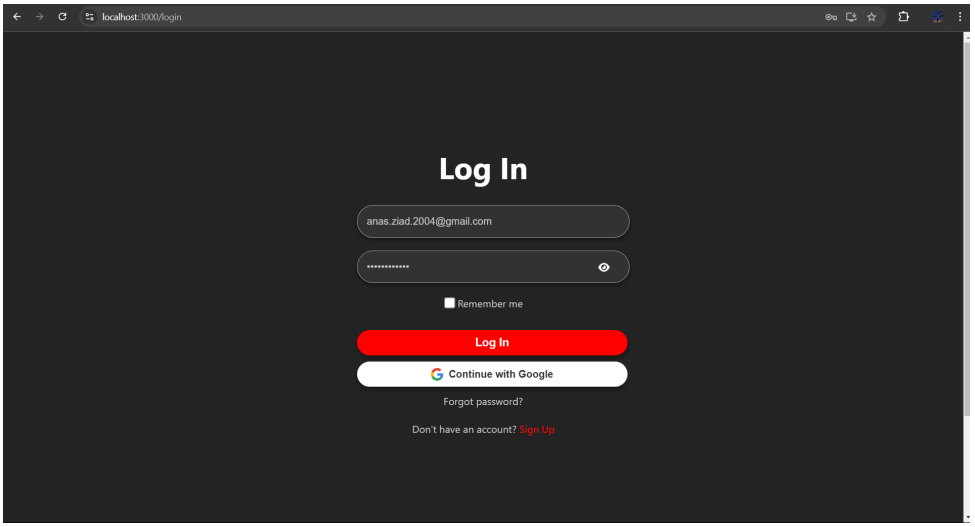


2- Login Page

The login page has input fields for :

- Email
- Password

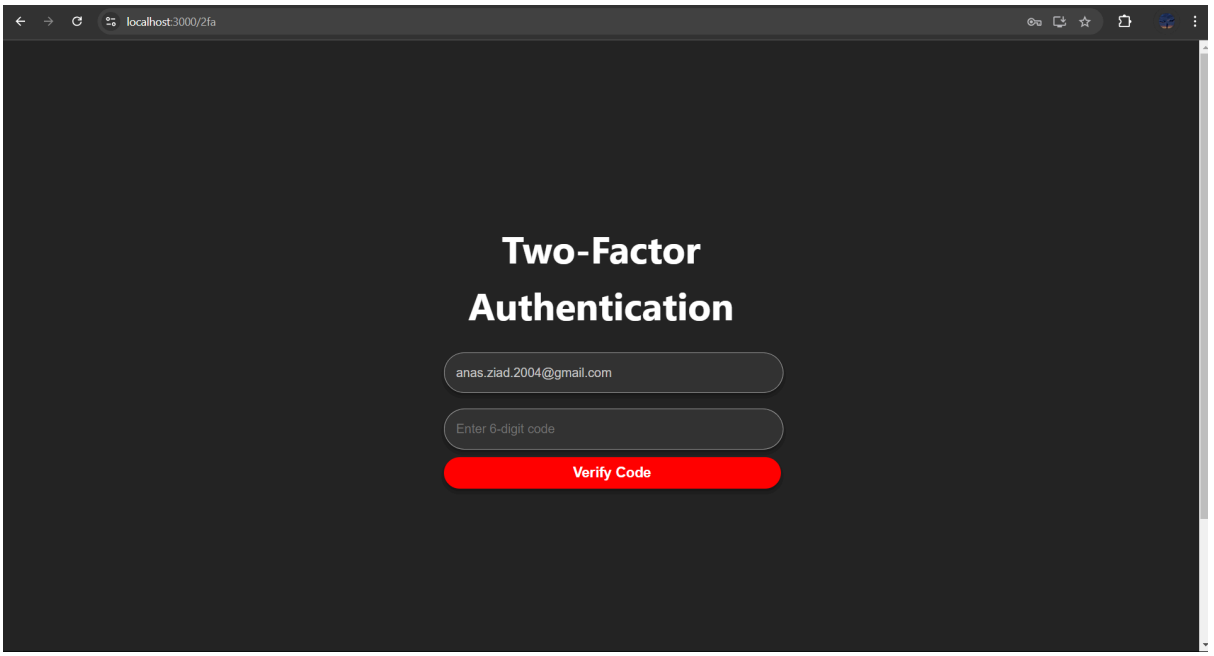
This page also has the option of logging in using the google account.



3- Two-Factor Authentication (2FA) Page

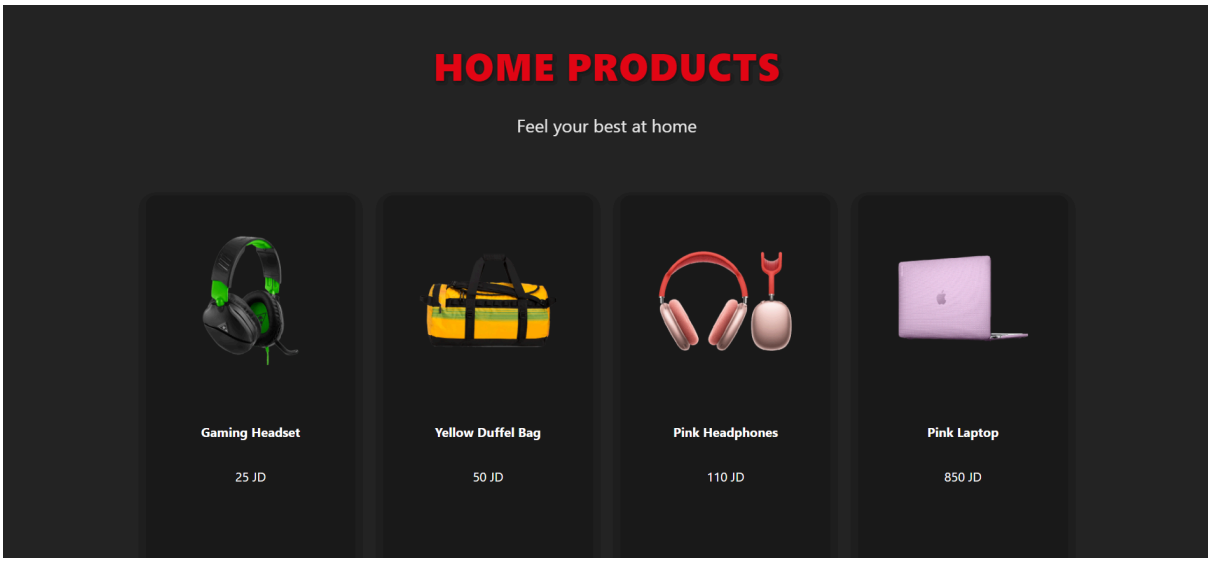
The 2FA page displays after a successful login and includes the following fields :

- Email field to show the email of the client
- Code field to enter a 6-digit code which will be sent via the email



4- Dashboard Page

After successfully logging in and authenticating using multi-factor authentication the client will gain access to the shop page which will display multiple images of the goods being sold with their prices.



Project requirements Implementation and Details

1- Input Validation and Sanitization

In our project, we have implemented input validation and sanitization to ensure data integrity and security, and we did take into consideration the following aspects.

Registration Page

A- Email Input Validation

This part of the code checks if the email entered by the user already exists in the database.

```
if (methods.length > 0) {
  return setError('Email is already in use');
}
```

B- Password Input Validation

- 8 length at least
- one uppercase
- one lowercase
- one number
- special char

C- Password Confirmation Validation

This section of the project compares the two fields of passwords entered by the client to ensure they match before proceeding with the sign-up.
If they do not match an error message will be shown to the client displays "Passwords do not match"

```
if (passwordRef.current.value !== passwordConfirmRef.current.value) {
  return setError('Passwords do not match');
}
```

D- Saving New User Data to the database

This section of the project saves the user data (Username , Email , DoB , User Password (Hashed)) to the database after a successful registration that meets all the requirements of the backend code

```
const userDocRef = doc(db, 'users', user.uid);
await setDoc(userDocRef, {
  name: nameRef.current.value,
  email: emailRef.current.value,
  dateOfBirth: dobRef.current.value,
  profilePicture: defaultProfilePicture,
  hashedPassword: hashedPassword,
});
```

Login Page

In the login page we programmed the backend code to handle user inputs as the following:

A- A variable will be declared which will be containing the email and password entered by the client in the login page and it will contain the cross site request forgery token provided by the server to the client.

B- The variable will check the data it holds with the database to ensure the validity of the inputs provided by the client

C- If the inputs are valid the server will send a verification link to the client before directing them to the two factor authentication page with there session data like email and cross site request forgery token , if not the site will firecet the client to the two factor authentication page but this time without there email and cross site request forgery token which will lead being directed to the login page to re-login.

```
  const response = await login(emailRef.current.value, passwordRef.current.value, csrfToken);

  if (response.requires2FA) {

    navigate("/2fa", { state: { email: emailRef.current.value } });
  } else {
    navigate("/2fa");
  }
}
```

2- CSRF Protection:

CSRF tokens are generated for all form submissions to prevent Cross Site Request Forgery attacks

The cross site request forgery token will be generated in beginning of every session as shown in the following code

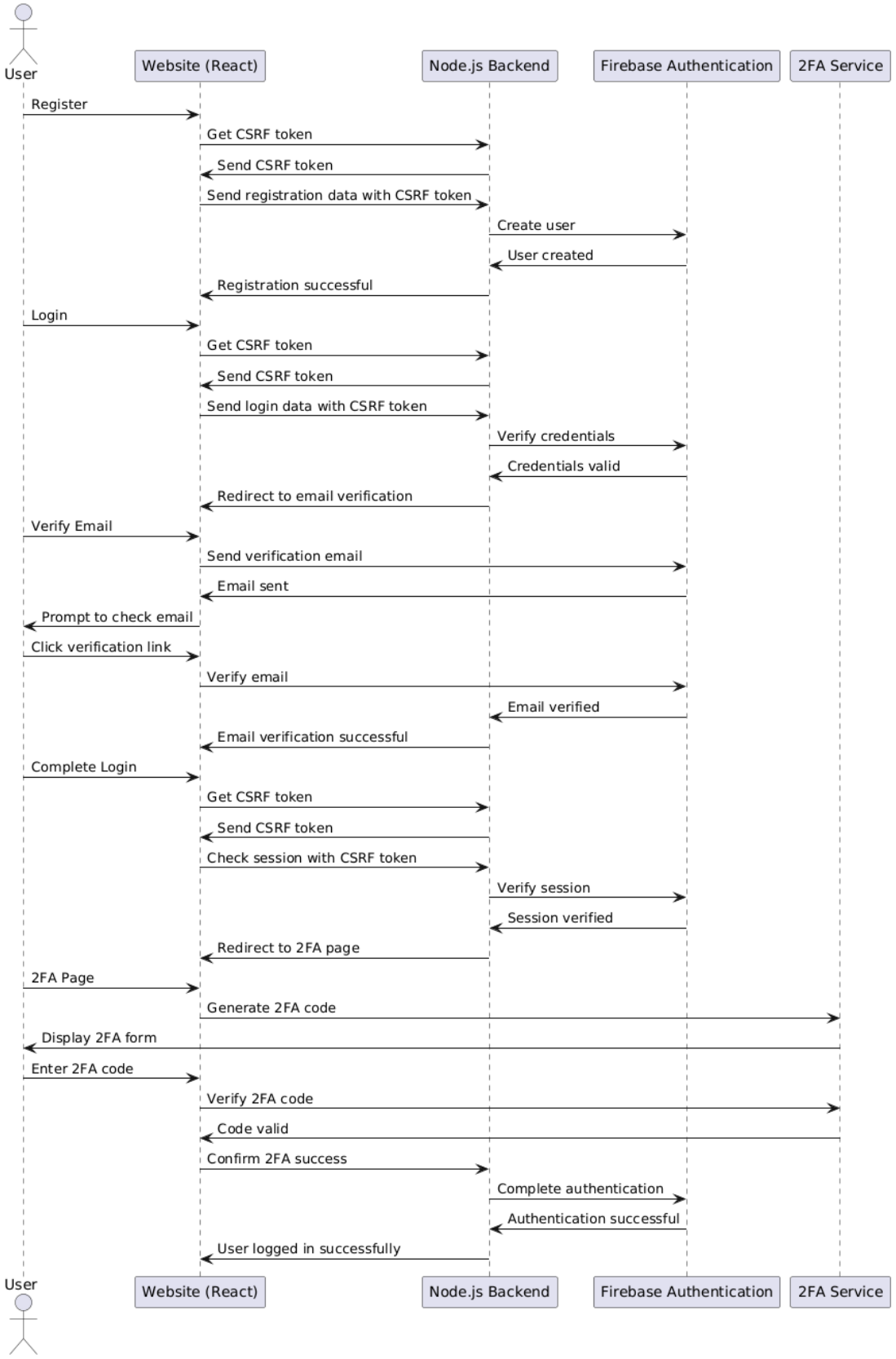
```
export function setCSRFToken(token) {
    if (!token) {
        throw new Error("Invalid CSRF token.");
    }
    localStorage.setItem('csrfToken', token);
    sessionStorage.setItem('csrfToken', token);
}
```

The token will be stored in the local storage of the database and it will be stored in the session storage which it's data will be cleared with the termination of the client's session

Then the server will check the validity if it's not valid the session will be terminated and if there was a malicious actor there session will be terminated as well.

```
export function validateCSRFToken(expectedToken) {

    const storedToken = sessionStorage.getItem('csrfToken') || localStorage.getItem('csrfToken');


    if (!storedToken || storedToken !== expectedToken) {

        terminateSession();
        throw new Error("Invalid CSRF token. Session terminated.");
    }

    return true;
}

export function terminateSession() {

    localStorage.removeItem('csrfToken');
    sessionStorage.removeItem('csrfToken');


}
```

CSRF


User
Website (React)
Node.js Backend
Firebase Authentication
2FA Service
Register
Get CSRF token
Send CSRF token
Send registration data with CSRF token
Create user
User created
Registration successful
Login
Get CSRF token
Send CSRF token
Send login data with CSRF token
Verify credentials
Credentials valid
Redirect to email verification
Verify Email
Send verification email
Email sent
Prompt to check email
Click verification link
Verify email
Email verified
Email verification successful
Complete Login
Get CSRF token
Send CSRF token
Check session with CSRF token
Verify session
Session verified
Redirect to 2FA page
2FA Page
Generate 2FA code
Display 2FA form
Enter 2FA code
Verify 2FA code
Code valid
Confirm 2FA success
Complete authentication
Authentication successful
User logged in successfully
User
Website (React)
Node.js Backend
Firebase Authentication
2FA Service

3- Secure Session Management:

In our project, we handle user session and authentication cookies from a JavaScript application. Very simplistically:

A. Managing Cookies

1. **setCookie**: It saves a cookie with a name, value, and settings (for how long).
2. **getCookie**: It reads a cookie based on its name.
3. **removeCookie**: It removes a cookie by its name.

B. Session Cookies

1. **setSessionCookie**: Saves a session cookie that persists for 1 hour.

C. Authentication Tokens

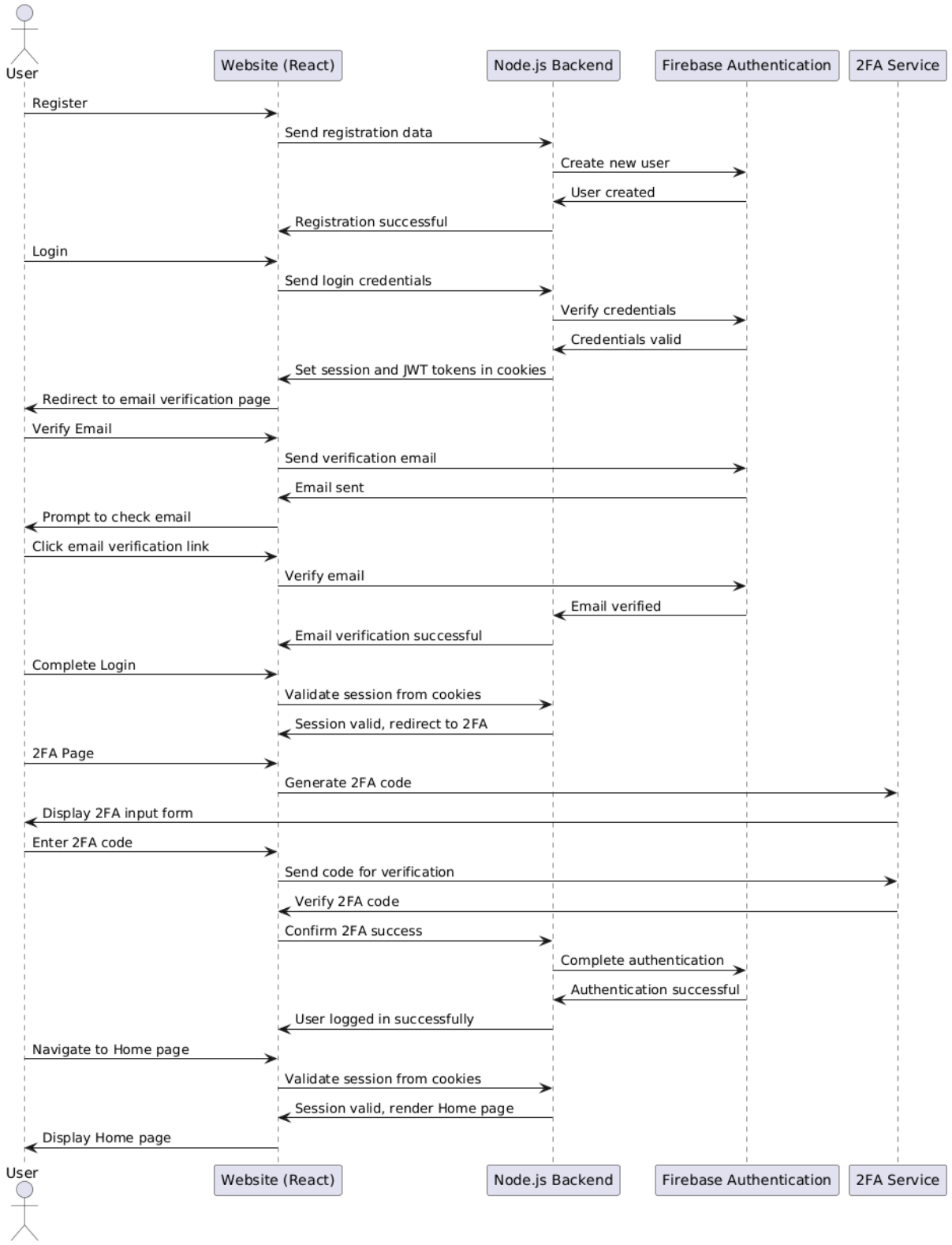
1. **setAuthTokens**: It saves two cookies:
2. **AccessToken**: Expiration time = 1 hour.
3. **RefreshToken**: 7 days valid.
4. **clearAuthTokens**: Clears both tokens and session cookies.

D. Session Validation

1. **validateSession**: Checks whether there is a session, aka session and accessToken cookies.

E. Refreshing Tokens

1. **refreshTokenIfNeeded**: If the accessToken expired, he gets a new one using the refreshToken. If that fails, he is logged out.


User
Website (React)
Node.js Backend
Firebase Authentication
2FA Service
Register
Send registration data
Create new user
User created
Registration successful
Login
Send login credentials
Verify credentials
Credentials valid
Set session and JWT tokens in cookies
Redirect to email verification page
Verify Email
Send verification email
Email sent
Prompt to check email
Click email verification link
Verify email
Email verified
Email verification successful
Complete Login
Validate session from cookies
Session valid, redirect to 2FA
2FA Page
Generate 2FA code
Display 2FA input form
Enter 2FA code
Send code for verification
Verify 2FA code
Confirm 2FA success
Complete authentication
Authentication successful
User logged in successfully
Navigate to Home page
Validate session from cookies
Session valid, render Home page
Display Home page
User
Website (React)
Node.js Backend
Firebase Authentication
2FA Service

Secure Session Management

4-HTTPS :

HTTPS is a secure extension of HTTP, the communication

protocol used to deliver web pages over the internet.

HTTPS is essentially HTTP with an added layer of security

provided by the Transport Layer Security (TLS) protocol.

While **HTTP** handles the transportation of data over the internet, TLS encrypts that data to ensure its security, giving rise to HTTPS. And we use this commands (to create and install SSL certificates for "localhost") :

bash$ npm install -g mkcert

bash$ mkdir certificates

bash$ cd certificates

bash$ choco install mkcert

bash$ mkcert localhost

bash$ mkcert -install

These set up HTTPS using a local SSL certificate and key

```
HTTPS=true
SSL_CRT_FILE=certificates/localhost.pem
SSL_KEY_FILE=certificates/localhost-key.pem
```

5- 2FA

This Node.js server offers the ability to enrol to Two-Factor Authentication (2FA) using email verification codes.

Key Features:

- **Send Verification Code**: A 6-digit code is emailed to users for verification purposes.
- **Verify Code**: Verifies the code and the email possessed by the user.
- **Rate Limiting**: Limits the amount of requests issued to prevent abuse (100 every 15 minutes).
- **Code Expiration**: Codes become invalid after a duration of 1 minute and are deleted.
- **Error Handling**: Feedback when data is missing, the code has expired or has been incorrectly entered.

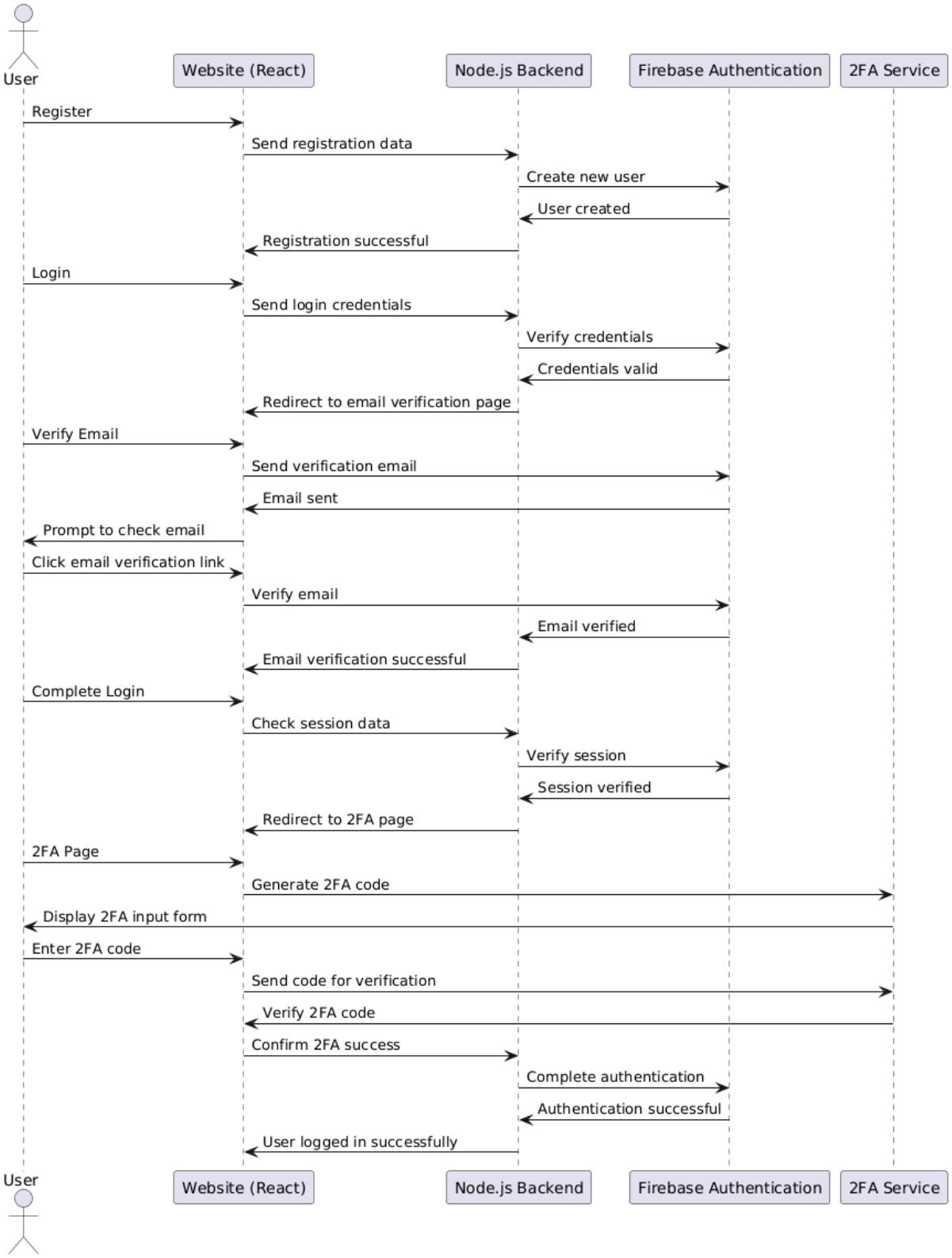
```javascript
const express = require("express");
const nodemailer = require("nodemailer");
const ratelimit = require("express-rate-limit");
const cors = require("cors");
require("dotenv").config();

const app = express();
const PORT = 3001;

app.use(express.json());
app.use(cors());
const limiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 100,
});

const verificationCodes = new Map();

const transporter = nodemailer.createTransport({
  service: "gmail",
  auth: {
    user: process.env.EMAIL_USER,
    pass: process.env.EMAIL_PASS,
  },
});

function cleanExpiredCodes() {
  const now = Date.now();
  for (const [email, data] of verificationCodes.entries()) {
    if (data.expires <= now) {
      verificationCodes.delete(email);
    }
  }
}
```

```javascript
app.post("/send-code", limiter, async (req, res) => {
  const { email } = req.body;

  if (!email) return res.status(400).json({ error: "Email is required" });

  const code = Math.floor(100000 + Math.random() * 900000).toString();
  const expires = Date.now() + 60000;

  verificationCodes.set(email, { code, expires });

  console.log(`Sending verification code to: ${email}`);
  console.log(`Verification code: ${code}`);

  try {
    await transporter.sendMail({
      from: process.env.EMAIL_USER,
      to: email,
      subject: "2FA Verification Code",
      html: `
        <h2>Your Verification Code</h2>
        <p>Code: <strong>${code}</strong></p>
        <p>Expires in 1 minutes</p>
      `,
    });
    res.json({ success: true });
  } catch (error) {
    console.error("Error sending email:", error);
    res.status(500).json({ error: "Failed to send verification code", details: error.message });
  }
});
```

```javascript
app.post("/verify-code", async (req, res) => {
  const { email, code } = req.body;

  if (!email || !code) {
    return res.status(400).json({ error: "Email and code are required" });
  }

  const storedData = verificationCodes.get(email);

  if (!storedData) {
    return res.status(400).json({ error: "No code found for this email" });
  }

  if (storedData.expires <= Date.now()) {
    verificationCodes.delete(email);
    return res.status(400).json({ error: "Verification code expired" });
  }

  if (storedData.code !== code) {
    return res.status(400).json({ error: "Invalid verification code" });
  }

  verificationCodes.delete(email);

  // Log to console when the code is verified
  console.log(`Verification successful for email: ${email}`);

  res.json({ success: true });
});

setInterval(cleanExpiredCodes, 60000);

app.get("/", (req, res) => {
  res.send("Server is running!");
});

app.listen(PORT, () => {
```

const email: any

2FA DIAGRAM


User
Website (React)
Node.js Backend
Firebase Authentication
2FA Service
Register
Send registration data
Create new user
User created
Registration successful
Login
Send login credentials
Verify credentials
Credentials valid
Redirect to email verification page
Verify Email
Send verification email
Email sent
Prompt to check email
Click email verification link
Verify email
Email verified
Email verification successful
Complete Login
Check session data
Verify session
Session verified
Redirect to 2FA page
2FA Page
Generate 2FA code
Display 2FA input form
Enter 2FA code
Send code for verification
Verify 2FA code
Confirm 2FA success
Complete authentication
Authentication successful
User logged in successfully
User
Website (React)
Node.js Backend
Firebase Authentication
2FA Service

6-Hashing function

```
+ Add field
    dateOfBirth: "2003-03-03"
    email: "fodesi4086@nongnue.com"
    hashedPassword: "$2a$10$rAvZetnZt4nm3NU1DMULI.Yda8RqpwGPJaPOL
    name: "fodesi4086@nongnue.com"
    profilePicture: "https://example.com/default-profile-picture.png"
```

Password Hashing is a process of converting an input password into a fixed-length string of characters, typically for the purpose of securely storing and transmitting the password. **Password hash functions** are designed to be one-way functions. This means it should not be computationally possible to reverse the process and get the original input password from the hashed value.
And we use :
Bcrypt is a type of cryptographic algorithm used to securely store passwords. It scrambles a user's password into a unique code. This way, even if a thief takes the database, they won't be able to recover the original passwords readily.

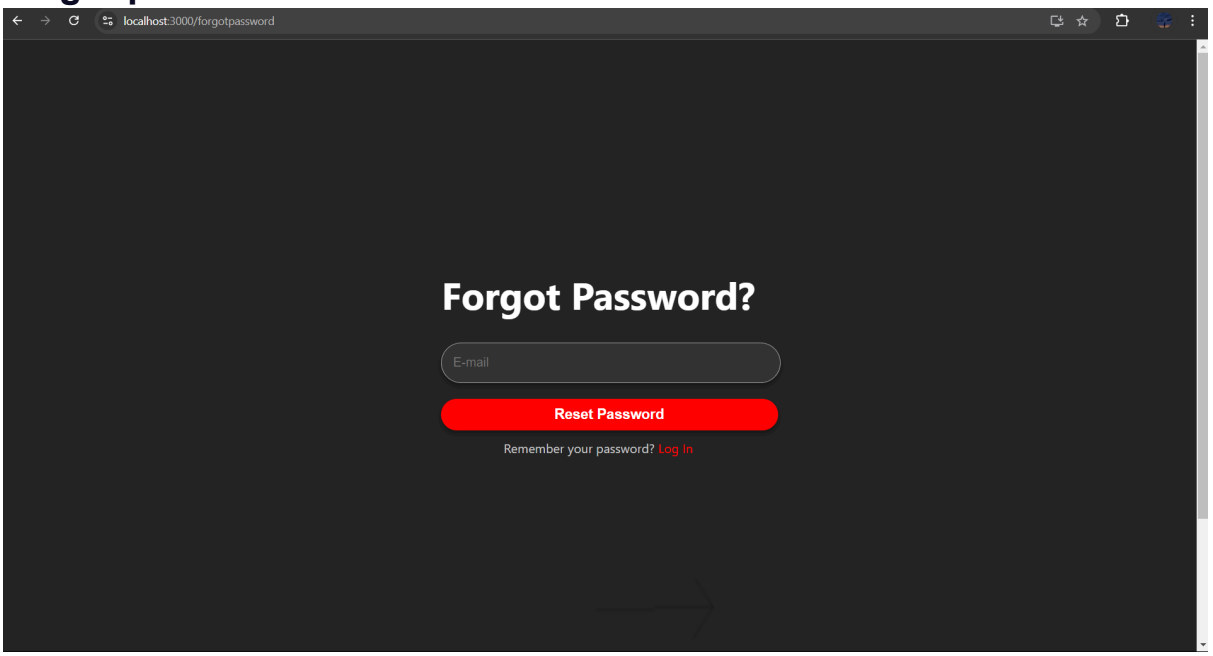
Const hashpassword= bcrypt.hashsync(password,10)

```
async function signup(email, password) {
  try {
    const hashedPassword = bcrypt.hashSync(password, 10);
    const userCredential = await createUserWithEmailAndPassword(auth, email, password);
    const user = userCredential.user;

    await sendEmailVerification(user);

    const csrfToken = getCSRFToken();
    await saveCSRFTokenToDatabase(user.uid, csrfToken);

    const userDocRef = doc(db, "users", user.uid);
    await setDoc(userDocRef, {
      email: user.email,
      wallet: 0,
      cartItems: {},
      name: user.displayName || null,
      dateOfBirth: null,
      profilePicture: user.photoURL || null,
      role: "user",
      hashedPassword: hashedPassword,
      requires2FA: false,
      emailVerified: false
    });
```

```
"$2a$10$rAvZetnZt4nm3NU1DMULI.Yda8RqpwGPJaPOLDl1.65cw5qhTvNxS"
```

Additional fetchers

Forgot password



he "Forgot Password" feature enables users to recover access to their accounts securely if they forget their login credentials.

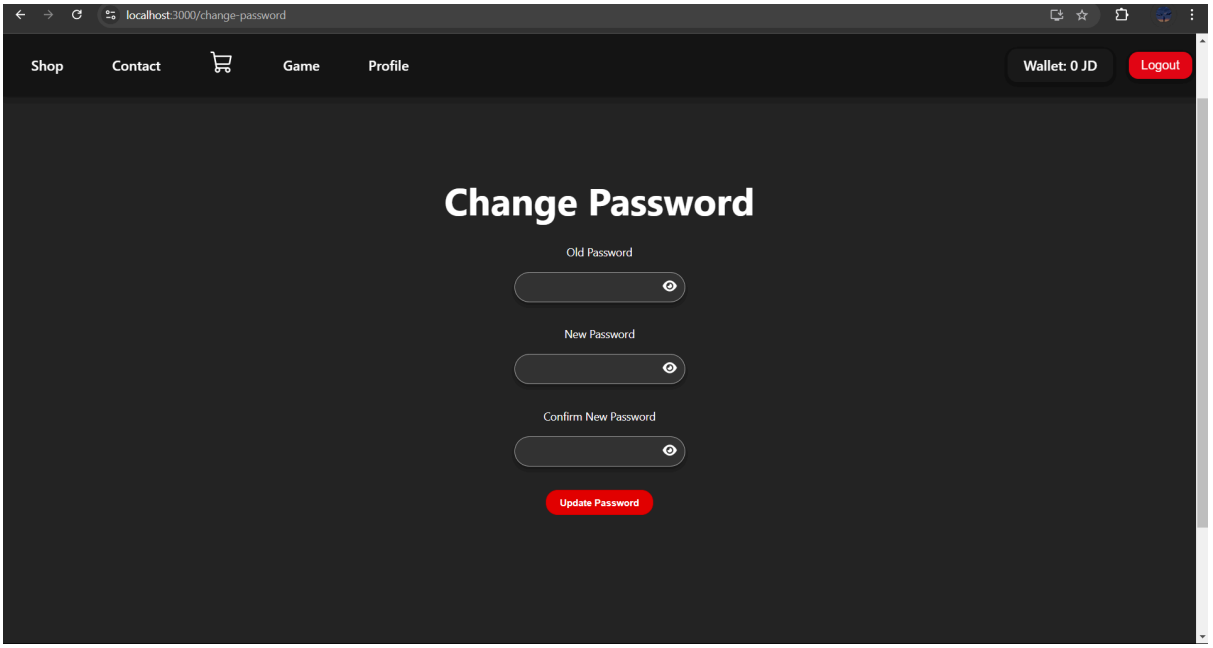
How it works

After clicking on the link *Forgot Password* he will be sent to a page that sends a link via his email to change the password.

```
import React, { useRef, useState,useEffect } from 'react';
import { useAuth } from '../User/AuthContext';
import { Link } from 'react-router-dom';



export default function ForgotPassword() {
  const emailRef = useRef();
  const { resetPassword } = useAuth();
  const [message, setMessage] = useState('');
  const [error, setError] = useState('');
  const [loading, setLoading] = useState(false);
  useEffect(() => {

    window.scrollTo(0, 0);
  }, []);
  async function handleSubmit(e) {
    e.preventDefault();

    try {
      setError('');
      setLoading(true);
      await resetPassword(emailRef.current.value);
      setMessage('Check your email inbox for further instructions.');
    } catch (error) {
      setError('Failed to reset password');
      console.error('Error sending password reset email:', error);
    }

```

```

  return (
    <div className="forgot-container">
      <div className="website-info">

        {}
      </div>
      <div className="forgot-box">
        <h2 className="forgot-text">Forgot Password?</h2>
        {error && <p className="error-message">{error}</p>}
        {message && <p className="success-message">{message}</p>}
        <form onSubmit={handleSubmit}>
          <div className="input-group">
            <input type="email" ref={emailRef} placeholder="E-mail" required />
          </div>
          <button type="submit" className="forgot-button" disabled={loading}>
            Reset Password
          </b
        </for (property) React.HTMLAttributes<HTMLDivElement>.className?: string | undefined
        <div className="login-link">
          Remember your password? <Link to="/login">Log In</Link>
        </div>
      </div>

    </div>
  );
}

```

Password update:



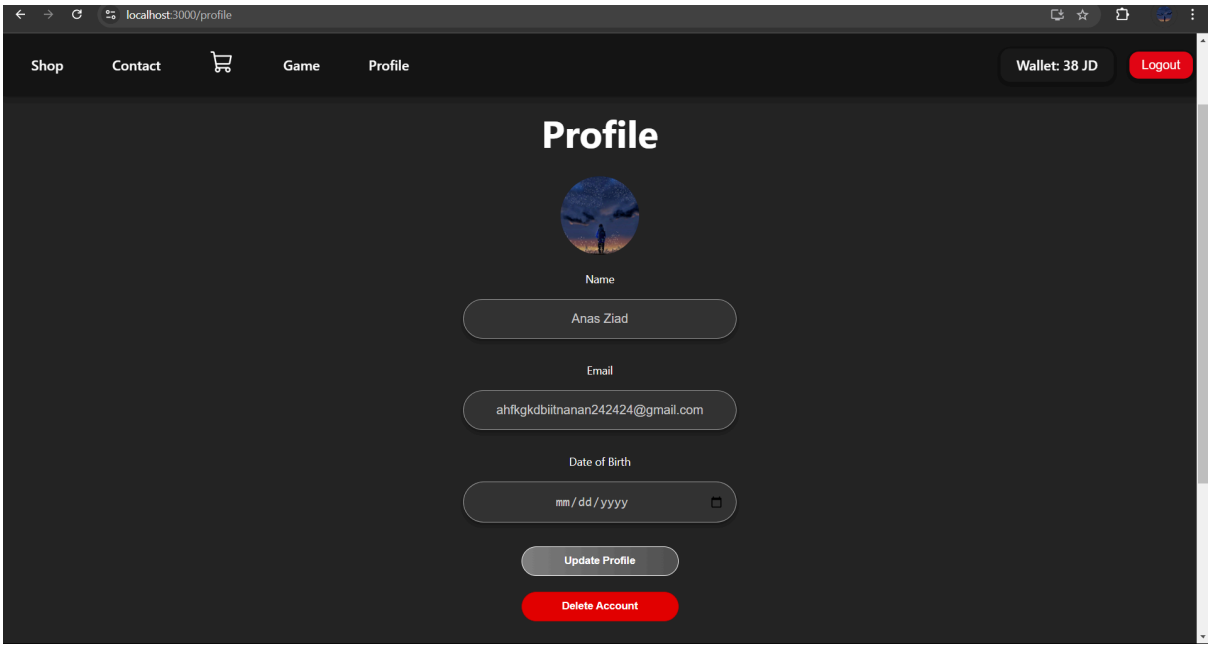
With the help of the “Change Password” function, users are able to change their passwords provisioned to them with utmost security.

Instructions:

1. Go to the “Profile” section.
2. Hit the button “Change Password”.
3. Input the current password followed by the fresh password you wish to use.
4. Please confirm your fresh password and submit your requests.

This is made possible so that every time the user feels that there is a need to update their password, it can be done and thus the account remains fully secure.

3. Profile



The “Profile” option is useful for users to check and edit their account details.

Main Features:

- View Details: For viewing in the user's name and surname, email address, as well as the date when the person was born.
- Update Profile: The users are able to alter their profiles’ information and store modified information.
- Enable Two-Factor Authentication: It ensures that the safety of the account and the associated mail is heightened.
- Change Email: Changes the current email address by the one that wants to be used.
- Change Password: Allows a user to change the password set.
- Delete Account: Sets the conditions for the user to remove his or her account entirely.

This feature abridges the capabilities of a user ensuring that updating and changing their account details is a smooth process.

(Admin panel will be on the database (firebase) no one but the admin can access)